

WISEConv: A Worklist-Driven Masked-Convolution Runtime for GPUs

Anonymous Author(s)

Abstract

Masked convolution promises to cut the per-frame inference latency of vision models by computing only positions an upstream policy marks active, yet existing paths sacrifice either throughput or useful-compute ratio and fail to convert sparsity into proportional latency reduction. We present WISEConv, a masked-convolution runtime that derives an active-output worklist from the upstream mask and uses it to drive the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by restricting computation to active positions while retaining the regular dense datapath. To amortize construction overhead and keep the compute pipeline fully utilized despite per-frame-varying worklist lengths, WISEConv co-designs construction and consumption: compatible operators reuse the constructed worklist without reconstruction, the worklist emission order preserves tile-local spatial reuse, and a data-driven adaptive decomposition adjusts parallel work to the activity level without host synchronization. Across four perception models and six GPU platforms (four discrete, two embedded), WISEConv reduces full-model latency by 22.1–46.5% relative to the fastest competing path and per-frame energy by 28.0–59.6% on the embedded platforms.

Keywords: Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

1 Introduction

Dense convolution dominates the per-frame inference latency of many latency-critical vision tasks [2, 9, 37], yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks active positions, and the operator computes only those. The mask may derive from event-camera increments [7, 22], temporal differences [16, 27], or learned spatial gates [20, 34]. Regardless of source, it poses one operator-level problem: compute the convolution only at the active positions of the dense feature grid.

Dense GPU convolution obtains high throughput from tiled, coordinate-driven execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position’s receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath. Neighboring coordinates share overlapping receptive fields, yielding predictable memory access and data reuse. However, unlike static sparsity in matrix multiplication, masked convolution selects positions at runtime per frame, requiring the operator to access scattered,

irregularly-overlapping neighborhoods and handle a varying workload, conflicting with tiled execution.

Existing masked-convolution systems preserve this regularity or eliminate wasted work, but not both. *Tile skipping* [7, 16, 27, 29] retains the dense pipeline and suppresses a tile only when all its outputs are inactive, so a single active position triggers computation for the entire tile. *Gather-scatter* [20, 34, 38] (as an execution path) instead extracts active positions exactly, materializes receptive fields, and scatters the computation results to the dense grid, sacrificing regularity and incurring extraction, irregular access, and writeback overhead.

We characterize this trade-off along two axes (§2.2): *throughput* (operations per second) and *useful-compute ratio* (the fraction of issued operations that produce needed outputs). In our sparsest workload (the DynConv-gated model, §6.2), the masks render 84.4% of the convolution work unnecessary, yet neither approach turns this into proportional latency reduction. Tile skipping [27] suffers a useful-compute ratio of only 49.1% and cuts latency by at most 40.6%. Gather-scatter drives the useful-compute ratio near one but sustains only 9.3% of tile skipping’s throughput, running 3.1× slower than the dense path.

Our key insight is that position-level selectivity does not require abandoning the dense convolution pipeline. A dense tile already derives its reads and writes from output coordinates, so replacing its contiguous coordinate block with an active-output worklist changes which outputs are computed without changing the per-position reduction, the dense tensor layout, or the weight layout. With this substitution, the operator runs as a dense tiled pipeline, raising the useful-compute ratio toward one without materializing input patches.

Translating this into latency reduction requires handling two challenges. First, *construction efficiency*: construction scans the mask, so its cost scales with the layer’s spatial grid, not with the active count or channel widths, while the compute it enables shrinks with both. Construction’s share of latency therefore grows as activity falls and channels thin (while enough positions remain active to keep the pipeline full), and accumulates across operators when every layer rebuilds. Second, *end-to-end worklist efficiency*: the worklist’s order and length both affect its consumption throughput. An unordered worklist scatters the input neighborhoods that adjacent work touches, lowering locality; its length falls below the dense grid and shifts with layer and input, so no fixed tiling keeps the GPU full, and because the length is device-resident, any host-side decision that depends on it

costs a synchronization on the per-frame path. Both effects sharpen as activity falls.

We present WISEConv (**W**orklist-**d**r**I**ven **ma**S**k**Ed **C**on**v**olution), a masked-convolution runtime that fuses position-level selectivity into the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. Construction is cheap and infrequent: a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse valid metadata instead of reconstructing it. Consumption is dense-like: construction emits contiguous per-tile segments ordered by the dense tile traversal, giving the compute stage tile-local structure without global sorting. Compute then decomposes parallel work adaptively according to device, layer shape, and activity, choosing each operator's configuration from offline calibration. For temporally correlated inputs, it tracks activity across frames and sustains GPU utilization without host synchronization.

This paper makes the following contributions:

- We characterize masked-convolution latency along two axes, useful-compute ratio and throughput, exposing why existing work does not translate upstream sparsity into proportional latency reduction.
- We design WISEConv, which fuses position-level selectivity into the dense tiled convolution pipeline and co-designs worklist construction and consumption to secure both axes jointly.
- We evaluate WISEConv on four perception models across four discrete GPUs and two Jetson platforms. WISEConv reduces full-model latency relative to the fastest baseline by 34.2–46.5% on the discrete GPUs and 22.1–38.6% on the Jetson platforms, and cuts per-frame energy over dense convolution by 28.0–59.6% on the Jetson platforms.

2 Background and Motivation

2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [2, 31, 35, 37, 41], where per-frame inference latency bounds system responsiveness [8–10, 17]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [7, 22], temporal residuals between video frames [3, 16, 26, 27, 36], or learned input-dependent gates [12, 20, 34, 38]. The event and temporal policies mark positions that changed since the previous frame; the learned gates mark positions the task treats as relevant. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution keeps the tensors and arithmetic of dense convolution and restricts only which output coordinates it computes. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{w} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size k , and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \dots, H_{out} W_{out}\}$, where each coordinate $p \in \mathcal{G}$ indexes a spatial position (h, w) and has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0, 1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0, 1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

M_{out} thus identifies the output positions that the execution path is required to compute, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{w}$. The active ratio $\alpha = \|M_{out}\|_0 / |\mathcal{G}|$, the active fraction of the grid, is the intrinsic sparsity the policy exposes.

An execution path issues computation over a sequence of position slots C , where each slot is drawn from $\mathcal{G} \cup \{\perp\}$; $\perp$ denotes a padding slot that carries no output coordinate. The path must satisfy the *coverage contract*: every active coordinate is included, $\{p : M_{out}[p] = 1\} \subseteq \{c \in C : c \neq \perp\}$. For each non- $\perp$ slot $p \in C$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{w}_i \mathbf{x}[\mathcal{N}_i(p)], \quad (2)$$

where $\mathcal{N}_i(p)$ is the i -th position in $\mathcal{N}(p)$ and $\mathbf{w}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. Positions the path does not compute retain their existing value in $\mathbf{y}$: masked convolution updates the output in place, so $\mathbf{y}[p]$ for $p \notin \{c \in C : c \neq \perp\}$ carries over from the previous frame (event and temporal policies) or from a one-time dense initialization (spatial gating). Coverage is therefore the correctness condition: every position the policy marks as changed is recomputed, and every other position is already correct by construction. Beyond coverage, a path may issue inactive or $\perp$ slots; dense convolution is the extreme where $\{c \in C : c \neq \perp\} = \mathcal{G}$. The policy fixes the accuracy and the required work, while C is determined by the path.

2.2 The Sparsity-to-Latency Gap

Existing systems execute a mask along one of two paths. *Tile skipping* [7, 16, 21, 26, 29, 36] inherits the dense pipeline's tile-based computation, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [27] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [20, 38] instead extracts exactly the active coordinates, computes, and scatters them back,

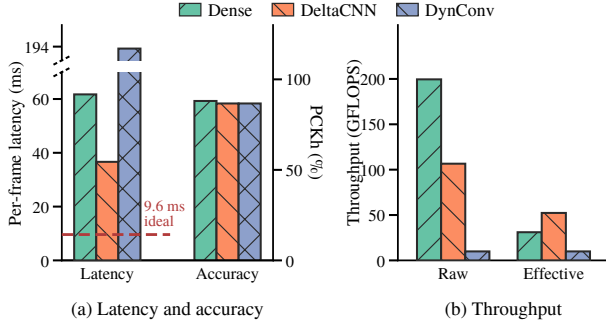


Figure 1. Statistics of DynConv [34], a pose estimator with learned spatial gating, on the MPII human-pose dataset [1] and Jetson AGX Orin, comparing the native cuDNN path (Dense) against the two sparse paths. (a) End-to-end average per-frame latency (left axis; the dashed line marks the dense latency scaled by the active ratio aggregated across layers) and PCKh accuracy (right axis; PCKh evaluates the percentage of keypoints localized within half the head size). The sparse paths apply the same mask, so their accuracy coincides. (b) Raw throughput T and effective throughput T_{eff} , aggregated across the model’s convolution layers. The dense useful-compute ratio equals the active ratio set by DynConv’s spatial gating ($\alpha \approx 15.6\%$).

removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [34] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

We measure each path by two quantities. Its useful-compute ratio is the fraction of issued position slots (including $\perp$) that carry an active output,

$$\eta = \frac{\|M_{\text{out}}\|_0}{|C|} \leq 1, \quad (3)$$

defined for $|C| > 0$. A path that issues no slots carries no ratio. An empty mask has $\eta = 0$ whenever the path nevertheless issues slots. Throughput is the rate at which a path processes issued FLOPs,

$$T = \frac{2|C|k^2C_{\text{in}}C_{\text{out}}}{t}, \quad (4)$$

where t is the per-frame latency, including all auxiliary work the path performs to enable computation (mask processing, coordinate generation, auxiliary passes). Because $|C|$ counts only the convolution’s issued position slots, auxiliary costs reduce T without changing η . The leading factor of 2 counts each multiply-accumulate as two FLOPs. When the required set is nonempty, latency for its $2\|M_{\text{out}}\|_0k^2C_{\text{in}}C_{\text{out}}$ FLOPs is set by the **effective throughput** $T_{\text{eff}} = \eta T$, the rate at which a path computes needed outputs.

Figure 1 traces this on Jetson AGX Orin for the sparsest workload in our evaluation. The policy marks 84.4% of the

convolution work unnecessary, and the sparse paths discard it with negligible accuracy loss, so the remaining work sets a 9.6 ms latency floor (Fig. 1a, dashed). Neither path approaches it: tile skipping (DeltaCNN [27]) reaches 36.6 ms, still $3.8\times$ the floor, and gather-scatter, even with DynConv built to amortize its overhead, runs at 193.5 ms, far slower than dense convolution itself. The gap traces to low effective throughput on both paths. Tile skipping holds throughput near half of dense but at a useful-compute ratio of only 49.1%; gather-scatter lifts the useful-compute ratio near one, yet extraction and data movement collapse its throughput to 5.0% of dense (Fig. 1b). Neither path efficiently converts the GPU’s capability into effective throughput for masked convolution.

2.3 Distinction from Related Sparse Execution

Related sparse-execution work differs from WISEConv’s masked convolution in representation, operator, sparsity type, or platform. 3D sparse convolution (SpConv v2 [30], TorchSparse++ [32, 33], Minuet [39], MinkowskiEngine [6], submanifold sparse convolution [15]) is built to organize computation over an unordered list of active voxels, constructing coordinate maps that route irregular input coordinates to outputs; its input is sparse by construction rather than a dense grid under a runtime mask, so it is related infrastructure rather than a baseline. Sparse matrix multiplication (DTC-SpMM [11], MaxK-GNN [28], Sputnik [13]) reorganizes a sparse matrix operand so the dense datapath can consume it; its sparsity lives in the matrix structure of a different operator, not in a spatial mask over a convolution grid. Structured weight sparsity (2:4 tensor cores [24]) and sparse-format compilers (TACO [19], SparseTIR [40]) exploit a static structural sparsity fixed at compile time in the weights or a known format, rather than a mask discovered at runtime over the spatial grid. DPACS [14] resolves similar dynamic spatial sparsity in CNN feature maps, but on a custom accelerator instead of a commodity GPU.

3 WISEConv Overview

Dense tiled convolution already consumes a stream of output coordinates: each coordinate determines the receptive-field reads and output write, while its reduction over channels and kernel offsets remains regular. WISEConv exploits this interface by replacing the dense coordinate sequence with a worklist of requested output positions, thereby introducing position-level selectivity into the tiled pipeline. Removing unrequested positions from the schedule raises the useful-compute ratio, but issuing fewer operations alone does not guarantee lower latency. To translate selectivity into latency reduction, the worklist must be inexpensive to construct and must be consumed with both a high useful-compute ratio and high throughput. WISEConv therefore co-designs a construction stage and a compute stage around these interdependent requirements, as Figure 2 summarizes.

[TODO] WISEConv overview. Left: the mask-space construction stage propagates the output mask, emits tile-local worklist segments into a preallocated buffer, and records the worklist length on the device; compatible operators reuse this metadata. Right: an occupancy-sized persistent grid consumes the device-bounded worklist, while the most recent completed input-activity observation from an earlier frame selects a high-throughput, low-padding workload decomposition.

Figure 2. The WISEConv two-stage pipeline. Construction operates only on mask and coordinate metadata, emits tile-locally ordered worklist segments, and reuses the resulting metadata across compatible operators. Compute bounds persistent work on the device and uses a completed, temporally lagged input-activity observation to adapt the workload decomposition without host synchronization.

Construction stage. To prevent worklist construction from erasing the savings of selective compute, WISEConv confines construction to mask and coordinate metadata, never reading feature or weight tensors. In one mask-space pass, it fuses the output-mask propagation in Eq. 1 with tile-local compaction, producing M_{out} and a worklist $\mathcal{W}$; the requested coordinates from each spatial tile form a segment in local dense-traversal order, preserving spatial grouping without global ordering. WISEConv stores $\mathcal{W}$ in a buffer preallocated for $|\mathcal{G}| = H_{out}W_{out}$ coordinates, the static upper bound on $|\mathcal{W}|$. A device-resident counter records $n_{\mathcal{W}} = |\mathcal{W}|$, so the host neither reads the active count nor allocates storage at runtime.

Even this metadata-only pass must scan the layer’s mask grid. WISEConv therefore propagates and reuses the resulting mask and worklist metadata across compatible operators instead of reconstructing it, amortizing one construction across the compatible operator sequence. The fused pass lowers the cost of each construction, while cross-operator reuse reduces how often it occurs; together, they address Construction Efficiency and Amortization.

Compute stage. The worklist length varies across layers and inputs and remains device-resident; synchronizing its current value to the host would stall the per-frame path, yet the workload decomposition must adapt because its padded work and exposed parallelism govern useful-compute ratio and GPU throughput. To execute without a host-visible bound, WISEConv launches only enough persistent thread blocks to fill the GPU’s concurrent block residency and lets them consume $\mathcal{W}$ through a grid-stride loop bounded by

device-resident $n_{\mathcal{W}}$, so the launch footprint tracks resident capacity rather than the full dense output grid.

To choose a decomposition without querying the exact $n_{\mathcal{W}}$, WISEConv builds from representative temporal traces a table that maps input-activity levels to configurations that sustain throughput while limiting padding, then indexes it with the most recent completed observation from one or more frames earlier; temporal correlation keeps this lagged signal informative, while completed-only polling prevents selection from blocking. The device count thus bounds the current work, while the data-driven signal chooses its decomposition; together, they sustain useful-compute ratio and GPU throughput as the worklist length varies without synchronizing the per-frame path.

Neither stage suffices alone: excessive construction cost, a low useful-compute ratio, or low consumption throughput can each erase the benefit of issuing less convolution work. By controlling all three jointly, WISEConv converts upstream spatial sparsity into end-to-end latency reduction.

4 WISEConv Design

The design follows the worklist from its construction and propagation across compatible operators (§4.1) to its consumption by the convolution pipeline (§4.2).

4.1 Worklist Construction and Reuse

4.1.1 Tile-Local Worklist Construction. Constructing a tight schedule already requires a scan of the output grid, but the resulting coordinates must also retain enough spatial

structure for efficient consumption. A globally stable compaction preserves the complete dense traversal order, but obtaining its device-wide offsets requires a prefix scan or an additional pass. Independent appends avoid that coordination, but scatter nearby positions across the schedule. Materializing active receptive fields, as gather-scatter does, regularizes the compute input at the cost of feature traffic proportional to the active count and $k^2 C_{in}$. The useful middle ground is therefore a metadata-only schedule that preserves order locally rather than globally.

WISEConv realizes this middle ground as globally unordered, locally ordered segments. Because the number of active positions is not known before the pass, it preallocates a coordinate buffer with capacity $|\mathcal{G}| = H_{out} W_{out}$, the static upper bound on $|\mathcal{W}|$, and initializes a device-resident counter for $n_{\mathcal{W}}$. The pass then partitions the output grid into $B_{\tau} \times B_{\tau}$ construction tiles τ . Within each tile, it applies the activity test of Eq. 1, writes the corresponding entries of M_{out} , and obtains a local active count n_{τ} . The tile reserves $[base, base+n_{\tau})$ from the counter and fills that interval with its active coordinates in local dense-traversal order (Algorithm 1). After all tiles complete, the reserved intervals collectively store $\mathcal{W}$, and the counter records $n_{\mathcal{W}}$. The layer shape statically determines both the buffer capacity and the construction launch geometry, while only the device observes the data-dependent worklist length. Construction therefore needs neither runtime allocation nor a host-visible active count.

The resulting device-resident metadata separates semantic demand from the execution schedule: M_{out} identifies the required positions, whereas $\mathcal{W}$ orders the coordinates scheduled by the worklist path. Reuse may make the scheduled-coordinate set a strict superset of the required set, but the coverage contract of §2.1 requires it to include every position marked by M_{out} .

This separation exposes where non-useful slots enter the worklist path. Construction and reuse determine how tightly $\mathcal{W}$ covers the required set, while compute may pad $\mathcal{W}$ when mapping it into C . For a worklist candidate with $n_{\mathcal{W}} > 0$, inserting the intermediate worklist length into Eq. 3 gives

$$\eta = \underbrace{\frac{\|M_{out}\|_0}{n_{\mathcal{W}}}}_{\eta_{\text{construct}}} \cdot \underbrace{\frac{n_{\mathcal{W}}}{|C|}}_{\eta_{\text{compute}}} . \quad (5)$$

The construction-side factor $\eta_{\text{construct}}$ is the fraction of $\mathcal{W}$ required by M_{out} , while the compute-side factor η_{compute} is the fraction of C occupied by worklist coordinates. These factors decompose η only; time spent constructing and consuming the worklist remains reflected in throughput T .

Fresh construction introduces no construction-side expansion. Its tiles partition the output grid, and their disjoint reservations place each active coordinate exactly once in $\mathcal{W}$. Consequently,

$$n_{\mathcal{W}} = |\mathcal{W}| = \|M_{out}\|_0, \quad \eta_{\text{construct}} = 1 \quad (n_{\mathcal{W}} > 0).$$

Algorithm 1 Tile-Local Worklist Construction

Require: Input mask M_{in} , output grid $\mathcal{G}$, tile size B_{τ}

Ensure: Mask M_{out} , worklist $\mathcal{W}$, length $n_{\mathcal{W}} = |\mathcal{W}|$

```

1:  $n_{\mathcal{W}} \leftarrow 0$ 
2: for each  $B_{\tau} \times B_{\tau}$  spatial tile  $\tau$  in parallel do
3:   for each position  $p \in \tau$  in parallel do
4:      $\text{active}(p) \leftarrow \exists r \in \mathcal{N}(p) : M_{in}[r] = 1$ 
5:      $M_{out}[p] \leftarrow \text{active}(p)$ 
6:   end for
7:    $n_{\tau} \leftarrow |\{p \in \tau : \text{active}(p)\}|$ 
8:    $\text{base} \leftarrow \text{ATOMICRESERVE}(n_{\mathcal{W}}, n_{\tau})$ 
9:   Write active  $p \in \tau$  to the reserved interval in traversal
   order
10: end for
```

Because coverage constrains membership rather than order, the order in which tiles reserve their intervals does not affect correctness. The within-interval traversal order is retained for locality.

The retained within-tile order is the spatial-structure contract passed to compute. Each segment follows dense traversal inside one bounded construction tile, keeping its coordinates' receptive fields nearby, while consecutive segments need not be spatially adjacent. Compute therefore inherits bounded spatial grouping within every segment it traverses without requiring a global order over $\mathcal{W}$.

Metadata-only construction reduces each pass to mask and coordinate work, but that work scales asymmetrically with the compute it enables. Receptive-field propagation scans all $H_{out} W_{out}$ output positions and performs $O(k^2)$ mask work per position, without scaling down with channel widths or the number of scheduled positions.¹ The required work of a fresh ungrouped convolution, by contrast, scales as $O(\alpha H_{out} W_{out} k^2 C_{in} C_{out})$. Construction consequently occupies a larger share of operator latency as activity falls or channels become thinner. Repeating the same grid scan at every same-grid operator would compound this channel-independent cost, which makes cross-operator amortization the next design requirement.

4.1.2 Cross-Operator Worklist Reuse. Once a spatial convolution has built a worklist, the same-grid operators that follow often add no new coordinate information. Convolution blocks commonly place 1×1 convolutions, activations, and inference-time normalization between operators that change the receptive field. Such operators either preserve the active set or narrow it, so reconstructing their worklists would only reproduce the incoming schedule or tighten an already valid superset. WISEConv avoids repeating the grid scan by making a compatible operator sequence, rather than an individual operator, the amortization unit of construction.

¹When an upstream policy supplies M_{out} directly, as in learned spatial gating [20, 34], construction compacts it without the k^2 receptive-field test.

Separating semantic demand from the reusable schedule makes inheritance safe. Alongside the feature tensor, WISEConv propagates M_{out} , $\mathcal{W}$, and device-resident $n_{\mathcal{W}}$. For operator i , let $\mathcal{G}_i$ be its output grid, write the worklist it consumes as $\mathcal{W}^i$ with length $n_{\mathcal{W}}^i = |\mathcal{W}^i|$, and define its active and scheduled sets as

$$\mathcal{A}^i = \{p \in \mathcal{G}_i \mid M_{out}^i[p] = 1\},$$

$$\mathcal{S}_{\mathcal{W}}^i = \{\mathcal{W}^i[m] \mid 0 \leq m < n_{\mathcal{W}}^i\}.$$

Fresh construction emits each coordinate at most once, and reuse leaves the worklist unchanged, so $|\mathcal{S}_{\mathcal{W}}^i| = n_{\mathcal{W}}^i$. The inherited schedule is valid for operator i whenever

$$\mathcal{A}^i \subseteq \mathcal{S}_{\mathcal{W}}^i \subseteq \mathcal{G}_i.$$

The first inclusion guarantees coverage, while the second keeps every coordinate in the downstream output space. Under this condition, operator i inherits the worklist and skips construction. Equality in the first inclusion gives tight reuse; strict inclusion gives superset reuse.

An active-set-preserving operator leaves both $\mathcal{A}^i$ and the inherited worklist unchanged. Same-grid 1×1 convolutions, activations that forward the mask, and inference-time normalization fall into this case. They preserve $\eta_{\text{construct}}^i = \eta_{\text{construct}}^{i-1}$ and retain tight reuse whenever the incoming schedule is tight. Reuse therefore removes a construction pass without expanding the schedule.

The same coverage relation admits reuse after a pointwise operator narrows the active set. WISEConv updates M_{out}^i but retains the incoming worklist, so $\mathcal{A}^i \subseteq \mathcal{S}_{\mathcal{W}}^i$ still holds and the schedule may become a strict superset. For $n_{\mathcal{W}}^i > 0$, the resulting construction-side factor is $\eta_{\text{construct}}^i = |\mathcal{A}^i|/n_{\mathcal{W}}^i$. Reuse trades the additional scheduled work for one fewer $O(H_{out}W_{out})$ construction pass. Thus preserving operators leave $\eta_{\text{construct}}$ unchanged, whereas narrowing operators can only decrease it within a compatible sequence.

A lower $\eta_{\text{construct}}$ does not itself trigger reconstruction. WISEConv rebuilds at statically identified semantic boundaries, where a change in the output grid may invalidate inherited coordinates or receptive-field expansion may introduce required positions outside the inherited schedule. The pass of §4.1.1 then restores $\mathcal{A}^i = \mathcal{S}_{\mathcal{W}}^i$ and starts a new compatible sequence. Within a compatible sequence, WISEConv retains the inherited schedule without observing per-operator activity; deciding dynamically whether a tighter schedule would repay another grid scan would reintroduce the cost that reuse removes.

The number of construction passes therefore follows semantic rebuild points rather than raw operator count. Each rebuilt worklist carries its tile-local structure through the following compatible sequence without another construction scan. Because its device-resident length still varies across sequences and frames, compute must execute a dynamic problem size.

4.2 Variable-Length Worklist Execution

For the worklist path, construction and reuse fix $\eta_{\text{construct}}$ before compute begins. The remaining factor arises as compute groups $\mathcal{W}$ into position tiles of B_M coordinates, with $\perp$ slots appearing only in the final tile. For $n_{\mathcal{W}} > 0$,

$$|C| = B_M \left\lceil \frac{n_{\mathcal{W}}}{B_M} \right\rceil, \quad \eta_{\text{compute}} = \frac{n_{\mathcal{W}}}{B_M \lceil n_{\mathcal{W}}/B_M \rceil}. \quad (6)$$

The final tile contains at most $B_M - 1$ padding slots, making this overhead most pronounced for a short worklist. The same choice of B_M also changes data reuse, block efficiency, and exposed parallelism, so maximizing η_{compute} need not maximize throughput T . The compute design must navigate this coupling while leaving the current $n_{\mathcal{W}}$ on the device.

4.2.1 Persistent Worklist-Driven Convolution. The worklist turns one fixed dimension of dense implicit GEMM into a device-resident runtime quantity. For a standard ungrouped batch-1 convolution, the dense mapping places output positions along rows M , output channels along columns N , and the channel and kernel reduction along depth K :

$$M = H_{out}W_{out}, \quad N = C_{out}, \quad K = k^2C_{in}.$$

Each tile computes $B_M \times B_N$ output elements and traverses K in B_K -wide steps. Because the dense problem dimensions (M, N, K) follow entirely from the layer shape, the backend can select (B_M, B_N, B_K) before execution. WISEConv preserves $N = C_{out}$ and $K = k^2C_{in}$ but replaces M with $n_{\mathcal{W}}$, which changes across invocations and remains unknown to the host. This creates two distinct problems: executing the current row extent without first exposing it to the host, and selecting a decomposition suited to that extent. We solve the execution problem here and defer selection to §4.2.2.

WISEConv changes only the source of each row coordinate. The dense path maps row m to the m -th coordinate p_m of the contiguous traversal ($0 \leq m < |\mathcal{G}|$); the worklist path instead reads $\mathcal{W}[m]$ ($0 \leq m < n_{\mathcal{W}}$). Each block loads a B_M -slot coordinate group, decodes the receptive-field bases of its non- $\perp$ entries, and computes their B_N output channels. The coordinates determine the input gathers and output stores, while the kernel retains the regular B_K -tiled reduction over K . It therefore reads directly from the dense feature tensor without materializing receptive-field patches. Processing $\mathcal{W}$ in contiguous ranges carries construction's within-segment grouping into these gathers.

Coordinate substitution defines individual rows, but their runtime count remains unknown to the host. WISEConv executes this device-resident row extent with an occupancy-sized persistent grid. At initialization, it uses each kernel configuration's concurrent residency to fix the corresponding block count. Selecting a configuration therefore selects a grid bounded by device capacity rather than $n_{\mathcal{W}}$. Once launched, the blocks read the current device-resident length

Algorithm 2 Persistent Worklist-Driven Implicit GEMM

Require: Device-resident worklist $\mathcal{W}$ and length $n_{\mathcal{W}} = |\mathcal{W}|$

Require: Input $\mathbf{x}$ and weights $\mathbf{w}$

Ensure: Output $\mathbf{y}$ (updated at worklist coordinates)

- 1: $n_{\mathcal{W}} \leftarrow \text{read device-resident length}$
- 2: {Dense implicit GEMM uses host-known $M = H_{out}W_{out}$.}
- 3: $n_{base} \leftarrow \lceil n_{\mathcal{W}}/B_M \rceil \lceil C_{out}/B_N \rceil$
- 4: **for** tile = block_id, block_id+grid_size, ... < n_{base} **do**
- 5: $(b_m, b_n) \leftarrow \text{Decompose}(\text{tile})$
- 6: **for** each row $0 \leq j < B_M$ **do**
- 7: $p_j \leftarrow \mathcal{W}[b_m B_M + j]$ if $b_m B_M + j < n_{\mathcal{W}}$
- 8: $p_j \leftarrow \perp$ otherwise
- 9: **end for**
- 10: {Dense sets $p_j \leftarrow p_{b_m B_M + j}$ and bounds against M .}
- 11: Decode each non- $\perp$ p_j to its receptive-field base address
- 12: Initialize accumulator $\mathbf{Acc} \leftarrow \mathbf{0}_{B_M \times B_N}$
- 13: **for** each B_K -wide reduction step over $K = k^2 C_{in}$ **do**
- 14: $\mathbf{x}_{tile} \leftarrow \text{gather for non-}\perp \text{ rows}$
- 15: $\mathbf{w}_{tile} \leftarrow \text{load corresponding weight slice}$
- 16: $\mathbf{Acc} \leftarrow \mathbf{Acc} + \mathbf{x}_{tile} \cdot \mathbf{w}_{tile}$
- 17: **end for**
- 18: Store $\mathbf{Acc}$ to $\mathbf{y}$ at the non- $\perp$ coordinates p_j
- 19: **end for**

and derive the base work-item count

$$n_{base} = \left\lceil \frac{n_{\mathcal{W}}}{B_M} \right\rceil \left\lceil \frac{C_{out}}{B_N} \right\rceil, \quad (7)$$

and cover all position and channel tiles with a grid-stride loop (Algorithm 2). The device-resident length determines the loop bound and final-tile padding, while the host launch remains independent of that length. The count n_{base} includes channel tiles because it counts thread-block work items; $|C|$ instead counts each issued position once and therefore has no channel-tile factor.

Sizing the grid from $n_{\mathcal{W}}$ on the host would synchronize at every operator. Sizing it for the full-grid upper bound avoids that readback, but submits blocks in proportion to the dense grid even when the worklist is short. WISEConv instead bounds submitted blocks by concurrent device residency and resolves the actual work only after launch. Persistent execution thereby handles any device-resident row extent; choosing the configuration that handles it efficiently remains a separate problem.

4.2.2 Trace-Calibrated Adaptive Decomposition. Persistent execution handles every worklist length, but the fastest candidate remains length-dependent because the decomposition jointly controls $\eta_{compute}$ and throughput T . For operator i , its active ratio and normalized worklist length

are

$$\alpha_i = \frac{|\mathcal{A}^i|}{|\mathcal{G}_i|}, \quad \lambda_i = \frac{n_{\mathcal{W}}^i}{|\mathcal{G}_i|}. \quad (8)$$

Fresh construction gives $\lambda_i = \alpha_i$, whereas superset reuse gives $\lambda_i \geq \alpha_i$. Because a worklist candidate processes the scheduled length rather than only the active set, WISEConv discretizes λ_i into levels ℓ , each represented by an absolute length $n_{i,\ell}$. Let Q_i be the candidates enumerated for operator i on the deployment device, and let $\hat{t}_i(q; n)$ be the measured latency of candidate q at length n . The worklist candidates in Q_i span (B_M, B_N, B_K, S_k) , where S_k partitions the reduction to expose additional parallelism. For a known level, the best candidate follows from standard autotuning:

$$q_{i,\ell}^* = \arg \min_{q \in Q_i} \hat{t}_i(q; n_{i,\ell}). \quad (9)$$

The candidate knobs affect $\eta_{compute}$ and T differently. B_M trades final-tile padding and the number of independent position tiles against intra-tile reuse and per-block efficiency. B_N and B_K tune the regular channel and reduction datapath without changing $|C|$. When position tiles alone expose too little parallelism, S_k partitions K and multiplies the work-item count in Eq. 7, also without changing $|C|$. This split can raise throughput on short worklists, but rereads coordinates and requires output zeroing and atomic accumulation. Calibration therefore measures every candidate-specific cost on worklists with the tile-local structure produced by construction, excluding only the construction pass shared by all candidates.

When full-grid execution preserves the operator semantics, Q_i also includes a dense candidate in the same measured-latency objective; it forwards the mask and worklist metadata unchanged for downstream operators.

Calibration therefore resolves $q_{i,\ell}^*$ offline. The remaining challenge is selecting ℓ at runtime without reading the current $n_{\mathcal{W}}$ back at every operator.

WISEConv avoids the readback by separating the current worklist bound from an approximate configuration signal. Persistent compute reads $n_{\mathcal{W}}$ on the device, so the host needs only a predictor of the corresponding level, not the exact bound. Event and temporal workloads provide one when two properties hold: their layer masks derive from a shared network-input mask through receptive-field propagation, and successive input masks correlate in time. Under these conditions, a completed observation of global input activity from an earlier frame provides a predictor of current per-operator lengths for decomposition selection.

Let M_0^f denote this network-input mask for frame f on grid $\mathcal{G}_0$, and let

$$\beta_f = \frac{\|M_0^f\|_0}{|\mathcal{G}_0|}, \quad b_f = \text{Bucket}(\beta_f).$$

WISEConv learns a per-operator mapping $\mathcal{M}_i$ from input bucket to worklist-length level. Combined with the candidate

table $\mathcal{L}_i[\ell] = q_{i,\ell}^*$, a completed observation selects

$$q_i(f) = \mathcal{L}_i[\mathcal{M}_i(\hat{b}_f)], \quad (10)$$

where $\hat{b}_f$ is the bucket derived from the most recent active-count transfer completed by the start of frame f . The observation may be one or more frames old, but it affects only the chosen configuration; the current device-resident $n_{\mathcal{W}}^i$ still bounds the worklist execution.

A second offline pass learns $\mathcal{M}_i$ from representative temporal traces. It first fixes the deployment reuse decisions, then records each frame's input bucket b_f together with the worklist length $n_{\mathcal{W}}^i(f)$ presented to every operator. Recording this schedule length, rather than only the cardinality of M_{out}^i , incorporates the superset schedules created by reuse. For each bucket, WISEConv maps the median normalized length to a level:

$$\mathcal{M}_i(b) = \text{Level}\left(\text{median}_{f: b_f=b} \lambda_i(f)\right). \quad (11)$$

The median targets the representative length rather than allowing a few outlying frames to determine the configuration. Each operator needs its own mapping because grid resolution, receptive-field size, and propagation depth transform the same input activity differently. The two calibration passes now compose directly: Eq. 11 predicts a worklist-length level from input activity, and Eq. 9 supplies the fastest measured candidate for that level.

The online loop applies this mapping using only completed observations (Algorithm 3). Each operator starts from a calibrated default. At the beginning of a frame, the runtime updates configurations from the most recent completed transfer, or retains them if none is available. It then generates M_0^f , enqueues an asynchronous transfer of its active count, and launches construction and compute. An enqueued observation becomes eligible only in a later frame; if the observation buffer is full, the runtime drops the new transfer. No step therefore waits for an activity observation.

Because every candidate satisfies coverage and each worklist candidate still reads the current $n_{\mathcal{W}}^i$, a stale, missing, or inaccurate observation can affect latency but not coverage.

Learned-gating models do not satisfy the shared-input-mask condition needed by the feedback mapping. Their network input remains dense, while each gated block derives its own mask from intermediate features, so global input activity conveys no information about per-operator worklist lengths. WISEConv assigns each such operator a fixed level calibrated from its gate statistics because tracking every gate would require one observation per operator rather than one per frame. Temporal workloads thus select calibrated candidates from completed observations, while learned-gating workloads retain fixed levels. In both cases, the current $n_{\mathcal{W}}$ bounds every worklist execution without synchronizing the per-frame path.

Algorithm 3 Trace-Calibrated Nonblocking Configuration Selection

Require: Candidate tables $\mathcal{L}_i$ and representative traces

Ensure: Per-operator candidates $q_i(f)$ without host synchronization

```

1: Offline trace mapping:
2: for each frame  $f$  of the traces do
3:    $b_f \leftarrow \text{Bucket}(\|M_0^f\|_0/|\mathcal{G}_0|)$ 
4:   for each operator  $i$  do
5:     Append  $n_{\mathcal{W}}^i(f)/|\mathcal{G}_i|$  to  $R_i[b_f]$ 
6:   end for
7: end for
8:  $\mathcal{M}_i(b) \leftarrow \text{Level}(\text{median } R_i[b])$  for all  $i, b$ 
9: Online (per sequence):
10:  $q_i \leftarrow$  calibrated default entry of  $\mathcal{L}_i$  for all  $i$ 
11: for each frame  $f$  do
12:   if an observation has completed then
13:      $\hat{b}_f \leftarrow$  bucket of the most recent completed observation
14:      $q_i \leftarrow \mathcal{L}_i[\mathcal{M}_i(\hat{b}_f)]$  for all  $i$ 
15:   else
16:     Retain current  $q_i$  {do not wait for a pending transfer}
17:   end if
18:   Generate  $M_0^f$  and its device-resident count
19:   Enqueue asynchronous count transfer; drop if the buffer is full
20:   Run each  $q_i$ ; worklist candidates use current  $n_{\mathcal{W}}^i$ 
21: end for

```

5 Implementation

We implement WISEConv as a CUDA C++ extension for PyTorch. The current backend executes FP32 tensors in channels-last layout and provides the mask-aware operators required by the evaluated networks: convolution, transposed convolution, pooling, pointwise activation, addition, concatenation, and upsampling. A model-conversion pass replaces compatible PyTorch modules while preserving the graph topology, convolution parameters, and learned weights. The upstream application continues to generate masks.

The construction kernel assigns contiguous output positions to each warp. A thread tests one receptive field and terminates once it finds an active input. A warp ballot identifies active lanes, population counts give their local ranks, and one atomic operation reserves a contiguous worklist segment for the warp. Active lanes then write their coordinates in lane order. This realizes the tile-local ordering in Section 4.1 without a device-wide prefix scan or sorting pass. An identity 1×1 convolution has the same input and output coordinate space; when its input already carries compatible mask and worklist metadata, the operator reuses that worklist and skips construction.

The compute kernel uses a persistent, tiled implicit-GEMM organization. It keeps partial sums in registers and stages input and weight tiles through shared memory. Supported architectures use asynchronous global-to-shared copies, with a synchronous path for earlier GPUs. Split- K exposes additional parallel work when the active-coordinate dimension alone cannot occupy the device. Transposed convolution follows the same output-driven organization, but partitions output coordinates by stride phase so that each phase visits only its valid kernel offsets.

Efficient scheduling depends jointly on the GPU, layer shape, and activity. We therefore autotune the sparse tile shape and split- K factor and cache the choice by device, convolution parameters, activity level, and execution policy. For layers whose state-update semantics permit dense evaluation, the candidate set can also include a cuDNN path. A sparse-only policy isolates the worklist-driven kernel, and the cache key distinguishes the two policies and the requested determinism mode. Autotuning and one-time weight preparation complete before the timed sequence described in Section 6.1.

For fixed input values, weights, and mask, WISEConv evaluates Eq. (2) at every active output coordinate. Floating-point reordering may produce small numerical differences from cuDNN, so we verify active outputs with numerical tolerances rather than requiring bitwise identity. This check separates backend correctness from the task-quality effect of the fixed upstream mask policy.

6 Evaluation

6.1 Experimental Setup

We evaluate WISEConv across four perception models, three mask sources, four GPU platforms, and three competing execution paradigms. The evaluation asks four questions: (1) does WISEConv reduce full-model latency relative to the fastest available path, (2) how effectively does it convert required-work reduction into latency reduction, (3) what overhead does worklist construction add, and (4) does replacing the masked-convolution backend change task output or accuracy?

6.1.1 Platforms. We evaluate four representative GPU platforms: two embedded SoCs (Jetson Xavier NX and Jetson AGX Orin), a laptop-class mobile GPU (RTX 4070 Laptop), and a desktop GPU (RTX 3080). These devices cover the hardware classes commonly used for onboard deployment and robotics development. The two Jetson systems anchor our target deployment setting, while the discrete GPUs test whether WISEConv generalizes beyond embedded SoCs.

6.1.2 Models, Tasks, and Mask Sources. Our selected workloads span three robotic perception tasks:

- **Event-based optical flow.** We evaluate FireFlowNet [25], a lightweight event-based optical-flow model that delivers high inference rates while retaining competitive accuracy, on MVSEC [42]. MVSEC contains event-camera sequences and corresponding ground-truth flow from indoor quadrotor flight and outdoor driving. Following Ev-Conv [7], consecutive 50-ms windows advance by 1 ms; events entering or leaving the window form the sparse increment from which we derive layer masks. We calibrate the sparsification policy on indoor_flying1 and evaluate on the other three sequences.
- **Video object detection.** We evaluate YOLOv8n and YOLOv8m [18] on MOT16 [23], a collection of crowded pedestrian videos with annotated person boxes. Both models produce bounding boxes and confidence scores, while their different scales expose how model size affects sparse-execution efficiency. DeltaCNN [27] supplies the temporal mask mechanism by propagating thresholded activation deltas between successive frames.
- **Human pose estimation.** We use the DynConv pose model [34] on MPII [1], which depicts people performing diverse activities and annotates their 2D body joints. The model predicts joint heatmaps; its learned spatial gates provide input-dependent masks at gated blocks.

Table 1 summarizes their models and evaluation scales. We use official weights for all four models [18, 25, 34]. For each workload, we fix these weights, inputs, and upstream mask policy; the sparse paths receive the same resulting layer masks and differ only in their masked-convolution backend.

Table 1. Evaluation workloads.

	FireFlowNet [25]	YOLOv8n/m [18]	DynConv Pose [34]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [7]	DeltaCNN [27]	DynConv [34]
Dataset	MVSEC [42]	MOT16 [23]	MPII [1]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,575	2,958

6.1.3 Execution Baselines. We compare WISEConv against three FP32 execution paths established by prior work.

- **Dense** uses the native PyTorch/cuDNN backend [5] with sparse execution disabled.
- **Tile skipping** follows the tile-sparse execution established by SBNet, Ev-Conv, and DeltaCNN [7, 27, 29].
- **Gather-scatter** uses the original DynConv CUDA backend for human pose [34] and an implementation equivalent to MMCV MaskedConv2d for FireFlowNet and YOLO [4].

6.1.4 Metrics and Methodology. Our primary metric is full-model GPU latency. The timed region includes mask propagation, worklist construction, all network operators, and output update, but excludes host-to-device input transfer, calibration, autotuning, and the one-time dense state initialization required by incremental models. We use CUDA events on the model's compute stream and process complete held-out sequences rather than isolated synthetic frames. Each sequence is repeated three times; we retain per-frame samples and report both aggregate latency and dispersion.

To explain latency, we report the required-work ratio ρ_{work} , useful-compute ratio η , convolution throughput, and the construction fraction of operator latency. Throughput here isolates the compute kernel; construction is reported separately as a fraction of operator latency, so readers can reconstruct the inclusive T of §2.2. We aggregate η as total active FLOPs over total scheduled FLOPs across all convolution layers, so each layer contributes in proportion to the work it actually schedules. Because layers differ in cost, an unweighted average of layer activities misstates the work a mask removes, so we weight each layer by its dense MAC count,

$$\rho_{\text{work}} = \frac{\sum_l \alpha^l W_{\text{dense}}^l}{\sum_l W_{\text{dense}}^l}, \quad \alpha^l = \frac{\|M^l\|_0}{H_{\text{out}}^l W_{\text{out}}^l}, \quad (12)$$

where W_{dense}^l is the dense MAC count of layer l . Task quality is measured with Average Endpoint Error (AEE) for optical flow, detection AP against the fixed evaluation references for YOLO, and PCKh for human pose. Accuracy comparisons use the same masks and operating point as latency measurements; they evaluate the upstream policy, while operator correctness is checked separately against dense convolution at every active output.

6.2 Full-Model Latency

6.3 Efficiency Analysis

6.4 Construction Overhead

6.5 Task Accuracy

References

- [1] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.
- [2] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.
- [3] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBinfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.
- [4] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyi Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).
- [5] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).
- [6] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [7] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. Ev-Conv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.
- [8] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739
- [9] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117
- [10] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712
- [11] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024–1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378
- [12] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.
- [13] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [14] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728
- [15] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [16] Amirhossein Habibi, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.
- [17] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838
- [18] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>
- [19] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc.*

- ACM Program. Lang. 1, OOPSLA (2017).
- [20] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.
- [21] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 422–435. doi:10.1145/3666025.3699348
- [22] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.
- [23] Anton Milan, Laura Leal-Taixé, Ian D. Reid, Stefan Roth, and Konrad Schindler. 2016. MOT16: A Benchmark for Multi-Object Tracking. CoRR abs/1603.00831 (2016).
- [24] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. CoRR abs/2104.08378 (2021).
- [25] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.
- [26] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. 17246–17255.
- [27] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.
- [28] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426
- [29] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.
- [30] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. <https://github.com/traveller59/spconv>.
- [31] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.
- [32] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [33] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.
- [34] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.
- [35] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.
- [36] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. 246–256.
- [37] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.
- [38] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.
- [39] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. 786–802.
- [40] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [41] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.
- [42] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer, Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo Event Camera Dataset: An Event Camera Dataset for 3D Perception. CoRR abs/1801.10202 (2018).